

Abstract:

The Graph-Based Movie Recommendation System is a full-stack web application designed to provide personalized movie recommendations using both NoSQL and graph database technologies. The system leverages MongoDB for scalable storage of user profiles, movie data, and interaction history, while Neo4j is used to model and analyze relationships between users and movies for intelligent recommendation generation.

The frontend is developed using React and styled with Tailwind CSS, providing a responsive and user-friendly interface for browsing movies and viewing recommendations. The backend is built using Node.js and Express, handling API requests and coordinating communication between databases.

Neo4j enables efficient graph traversal to identify patterns such as similar users, shared interests, and connected movie genres. By combining content-based and collaborative filtering techniques, the system generates accurate and real-time recommendations.

This project demonstrates the integration of modern web technologies with hybrid database architecture to solve real-world recommendation system challenges.

Problem Statement:

Traditional recommendation systems often rely on relational databases, which struggle to efficiently handle complex relationships between users and items. This results in slower query performance and less accurate recommendations.

Key challenges include:

- Efficiently managing large-scale user and movie data
- Modeling complex relationships such as user preferences and similarities
- Generating real-time recommendations with low latency
- Ensuring scalability as data volume grows
- Delivering an intuitive and responsive user experience

This project addresses these challenges by combining MongoDB for flexible data storage and Neo4j for high-performance graph-based relationship analysis.

Objective:

The primary objectives of this project are:

- To design and implement a web-based movie recommendation system
- To store and manage user and movie data using MongoDB
- To model relationships between users and movies using Neo4j
- To implement graph-based recommendation techniques using Cypher queries
- To generate personalized recommendations based on user behavior
- To develop a responsive frontend interface using React
- To build a scalable backend using Node.js and Express
- To demonstrate hybrid database architecture in real-world applications

Implementation:

The implementation is divided into two primary components: the Frontend (React/Vite) and the Backend (Node.js/Express). The following sections describe the key implementation details.

System Architecture:

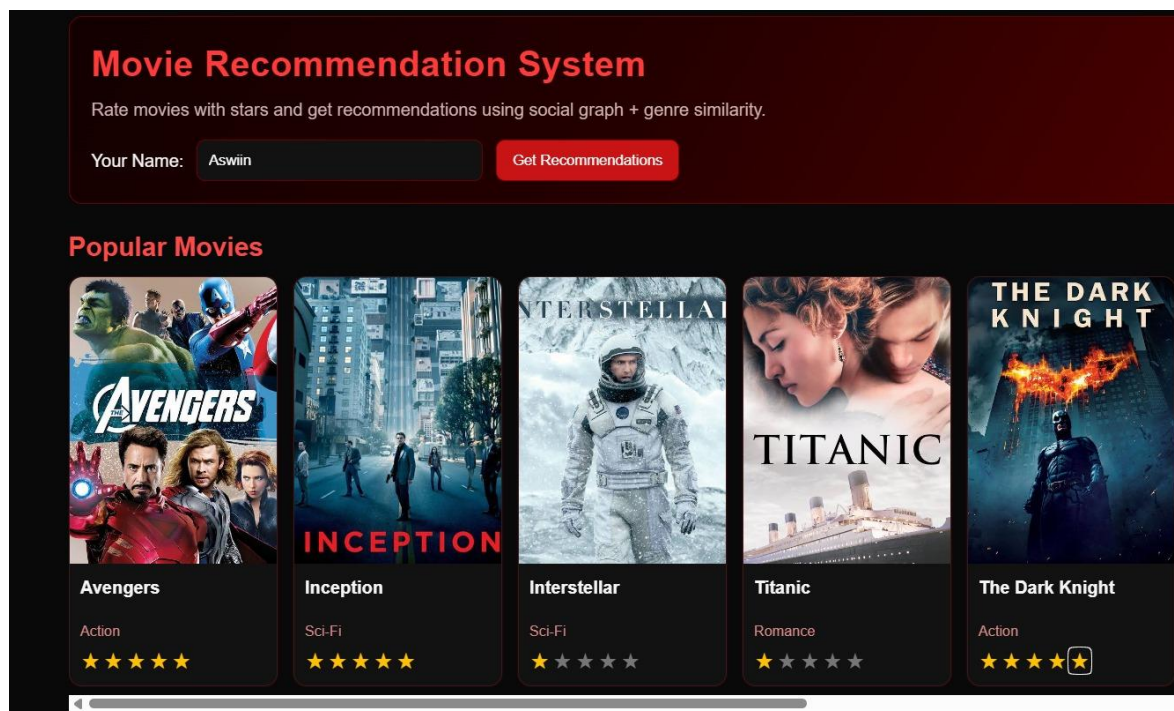
- The React frontend communicates with the backend using REST APIs
- The Node.js backend processes requests and handles database operations
- MongoDB stores structured data such as users, movies, and ratings
- Neo4j manages graph relationships for recommendation logic

Tech Stack Overview:

Layer	Technology	Purpose
Frontend	React (Vite)	Component-based UI framework
Frontend	Tailwind CSS	Styling
Backend	Node.js+Express	Server and API
Frontend	MongoDB(mongoose)	Data Storage
Frontend	Neo4j	Graph Relationships
Backend	Neo4j Driver	Graph database connection

Frontend Implementation:

- Developed reusable React components for movie listing and recommendation display
- Integrated API calls to fetch movie data and recommendations
- Designed responsive UI using Tailwind CSS
- Provided interactive features such as rating and browsing



Backend Implementation:

- server.js: Initializes Express server and database connections
- config/: Contains MongoDB and Neo4j connection setup
- models/: Defines schemas for users, movies, and ratings
- routes/: Handles API endpoints for movie operations and recommendations

Graph-Base Recommendation Logic

Socket.io enables persistent WebSocket connections. When a player makes a move, the frontend emits a move event with the updated FEN string. The server validates the move, updates game state, and broadcasts to the opponent. Disconnection events are handled gracefully.

Neo4j is used to model relationships such as:

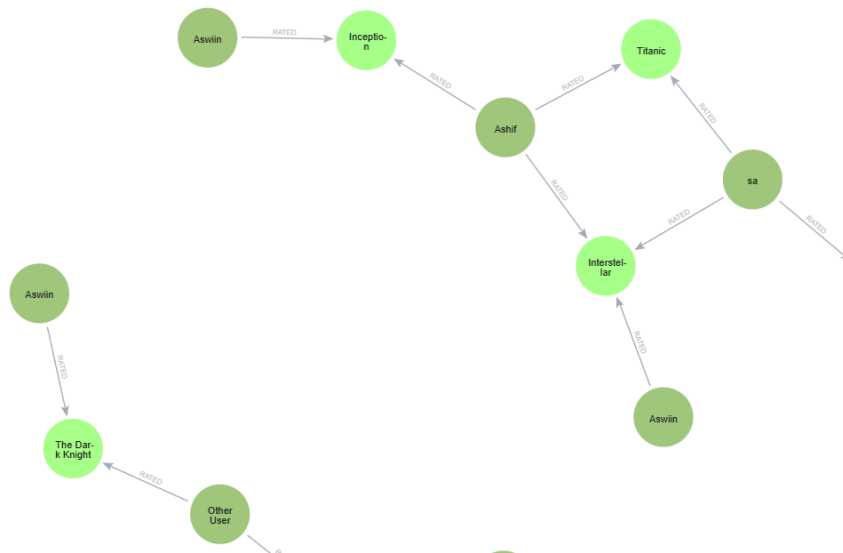
- User → WATCHED → Movie
- Movie → BELONGS_TO → Genre
- User → SIMILAR_TO → User

Using Cypher queries, the system:

- Identifies users with similar viewing patterns
- Recommends unseen movies to the current user

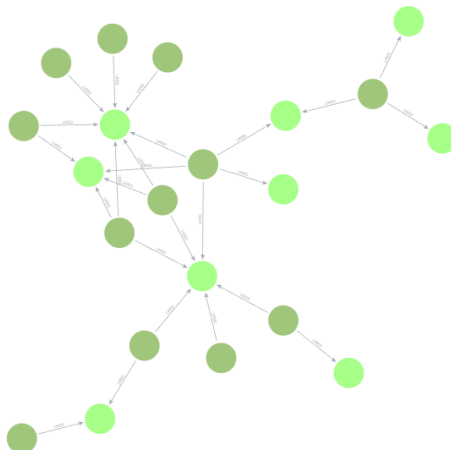
Example logic:

- If User A and User B watched similar movies
- Recommend movies watched by User B to User A



Data Flow

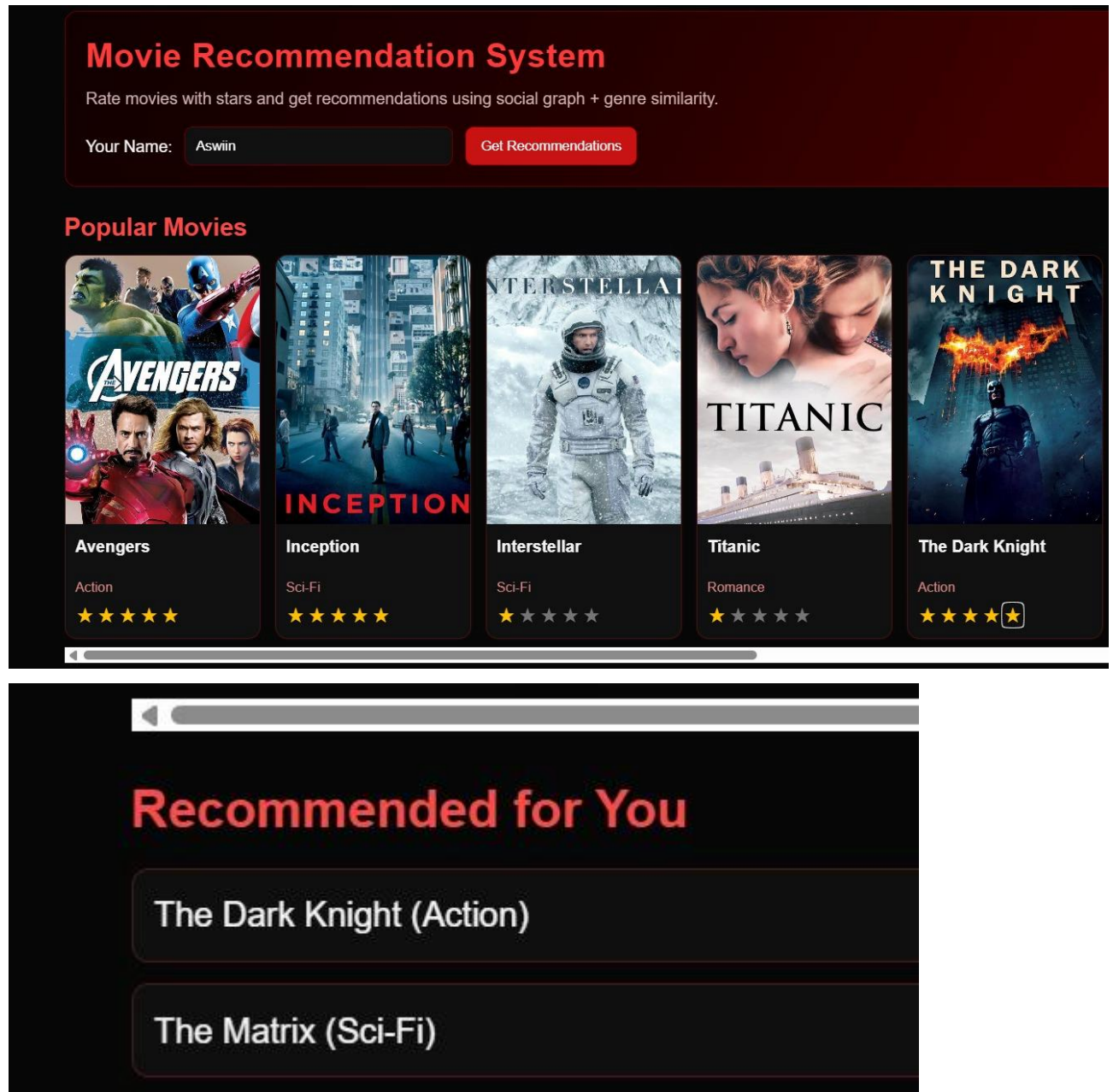
- > User interacts with frontend (rates/watches movies)
- > Data is stored in MongoDB
- > Relationships are updated in Neo4j
- > Neo4j processes graph queries
- > Recommendations are sent to frontend



Result

The Graph-Based Movie Recommendation System was successfully implemented and tested. The following results were achieved:

- Accurate personalized recommendations based on user behavior
- Efficient graph traversal enabling fast recommendation generation
- Seamless integration of MongoDB and Neo4j
- Responsive frontend with smooth user interaction
- Scalable backend capable of handling multiple users



Conclusion

The project successfully demonstrates the use of hybrid database architecture by combining MongoDB and Neo4j. The system efficiently handles large-scale data while providing accurate and real-time recommendations using graph-based analysis.

This project serves as a strong foundation for further enhancements such as:

- Real-time recommendation updates
- Integration with external movie APIs
- Advanced machine learning models
- User preference learning and ranking systems

